

Chapter 3

Concept of Flow

Control Statements

in Python

Introduction

In day-to-day life, we first identify the alternatives and choose between them, based on the values, beliefs, and preferences. The action of making very important decisions is called decision-making. Sometimes, the decisions are in favor of us and sometimes they are not. However, our journey of struggle continues always and forever. We anticipate conditions during our journey and make decisions. Then we decide which path to choose, in order to reach to our target. The same analogy of situations arises in programming, where we also need to make decisions, and based on the decision, we will execute the block of code. The direction of flow of execution of the program is decided by the decision-making statements in programming languages.

Structure

In this chapter, we will cover the following topics:

- Flow of execution of the program
 - o Selection statements/conditional statements

- o Iterative statements

- o Transfer increments

74 ● *Python for Everyone*

- In keyword usage

- Loop patterns

Objectives

By the end of this chapter, the reader will understand about conditional, iterative and transfer statements, along with the application of 'in' keyword in their Python code snippet. The reader will also understand the flow of control while executing these statements in their code. Different patterns such as star, alphabet or number will be displayed using loops and print statement.

Flow of execution of the program

The order in which the code statements are going to be executed at run time, is decided by the flow control. In Python, the flow control statements are divided into 3 types:

- Selection statements/conditional statements
- Iterative statements
- Transfer statements

Selection statements/Conditional statements

The selection or conditional statements are **if**, **if-else** and **if-elif-else**.

if

One of the simplest decision-making statement is the **if** statement. If a certain condition is met, then a statement or block of statements will be executed. Otherwise, they will not be executed. The syntax of the preceding statement is: **if test expression:**

statement(s)

The test expression after program evaluation will be either True or False, that is, Boolean values are accepted. If True, then a block of statements will be executed, otherwise not. The non-zero values will be interpreted as True. 0 and None will be interpreted as False. The test expression can be used with a bracket also. To identify a block, Python uses indentation. Let us see an example: if test expression:

Concept of Flow Control Statements in Python 75

statement1

statement2

If the test expression is True, then **if** block will consider only statement1 to be inside its block. The flowchart of **if** statement is shown in *Figure 3.1*:

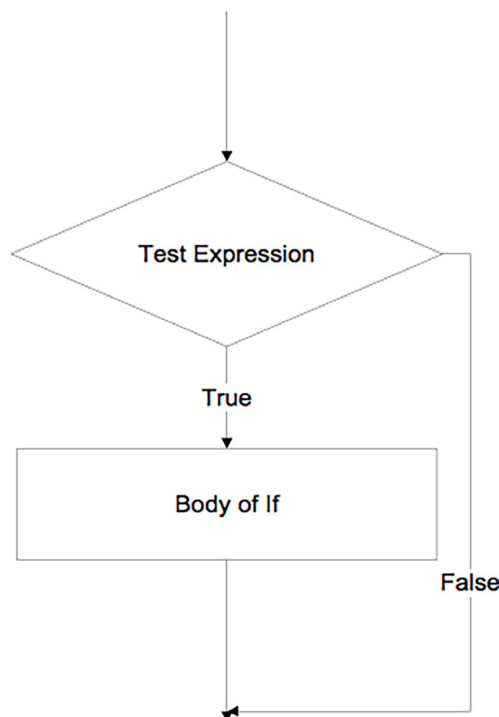


Figure 3.1: Flowchart of if statement

Let us clear the concept of **if** statement with the following example: `my_age = int(input("Enter your age:"))`

```
if my_age == 32:
```

```
    print("I am 32 years old")
```

```
print("Good Morning!")
```

Output:

Scene-1

Enter your age: 32

I am 32 years old

Good Morning!

76 ● *Python for Everyone*

Scene-2

Enter your age: 33

Good Morning

Note: The preceding code is covered in (Program Name: Chap03_Example3.1.py)

In the preceding program, the user will be prompted to enter the age. **my_age == 32** is the test expression. The statement inside **if** will be executed if the user enters the age as 32. If the user enters a different age from 32, then the preceding test expression will be False and the statement of code inside **if** statement will be skipped. The **print("Good Morning")** method falls outside of the **if** statement (un-indented). So, it will be executed, regardless of the test expression to be True or False.

The flow chart of the preceding code is shown in *Figure 3.2*:

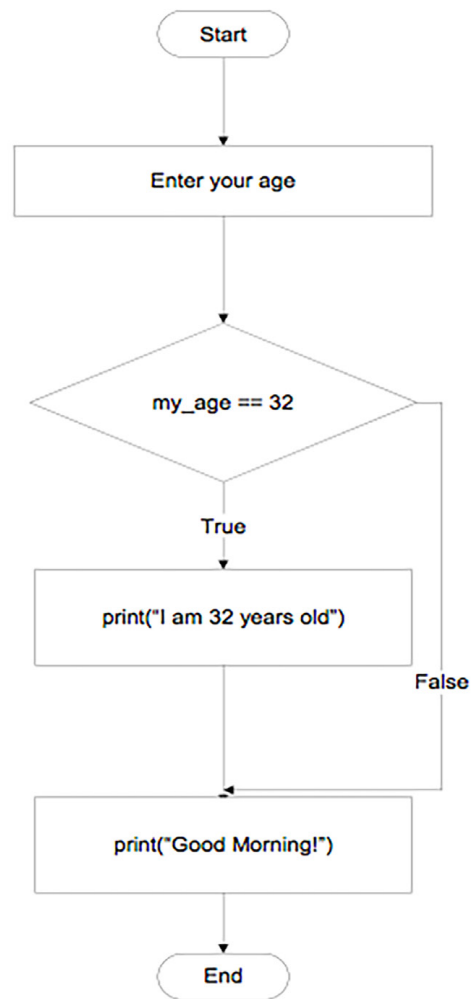


Figure 3.2: Flowchart for if code

In Scene-1, the user has entered the age as 32. So, the **print()** statement inside **if** test expression is executed, followed by the print statement outside of the **if()** block.

In Scene-2, the user has entered different age than 32. So, the **print()** statement outside of the **if()** block will be executed.

if-else

In the previous statement, we came to know that if a test expression is True, then a block of statements is executed. If False, then it would not be executed. But we can do something else if the test expression is False. When the test expression is False, the **else** statement is used with **if** statement to execute a block of code. The syntax of the preceding statement is as shown:

if test expression:

Body of if

else:

Body of else

When the test expression is True, body of **if** will be executed. If False, then body of **else** will be executed. To separate the blocks, indentation is used. The flow chart of **if-else** statement is shown in *Figure 3.3*:

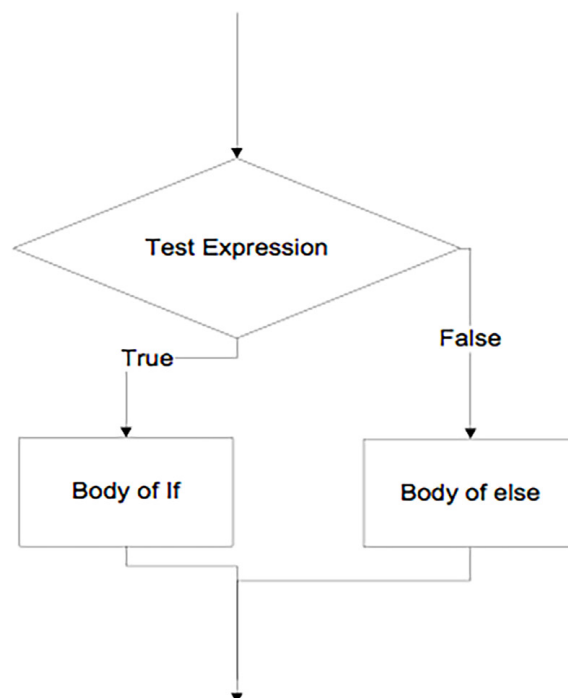


Figure 3.3: Flowchart of if-else statement 78 ● Python for Everyone

Refer to the following program:

```
num = int(input("Enter a number: "))

if num%2 == 0:

    print("Even number")

else:

    print("Odd number")
```

Output:

Scene-1

Enter a number: 12

Even number

Scene-2

Enter a number: 13

Odd number

Note: The preceding code is covered in (Program Name: Chap03_Example3.2.py)

In the preceding Python snippet code, the user will be prompted to enter the number to check whether the entered number is even or odd.

In Scene-1, the entered number is 12. The test expression **num%2 == 0** is True, and hence, the body of **if** will be executed and output “**Even number**” will be displayed.

In Scene-2, the entered number is 13. The test expression **num%2 == 0** is False, and hence, the body of **else** will be executed and output “**Odd number**” will be displayed.

The flow chart of the preceding example is shown in *Figure 3.4*:

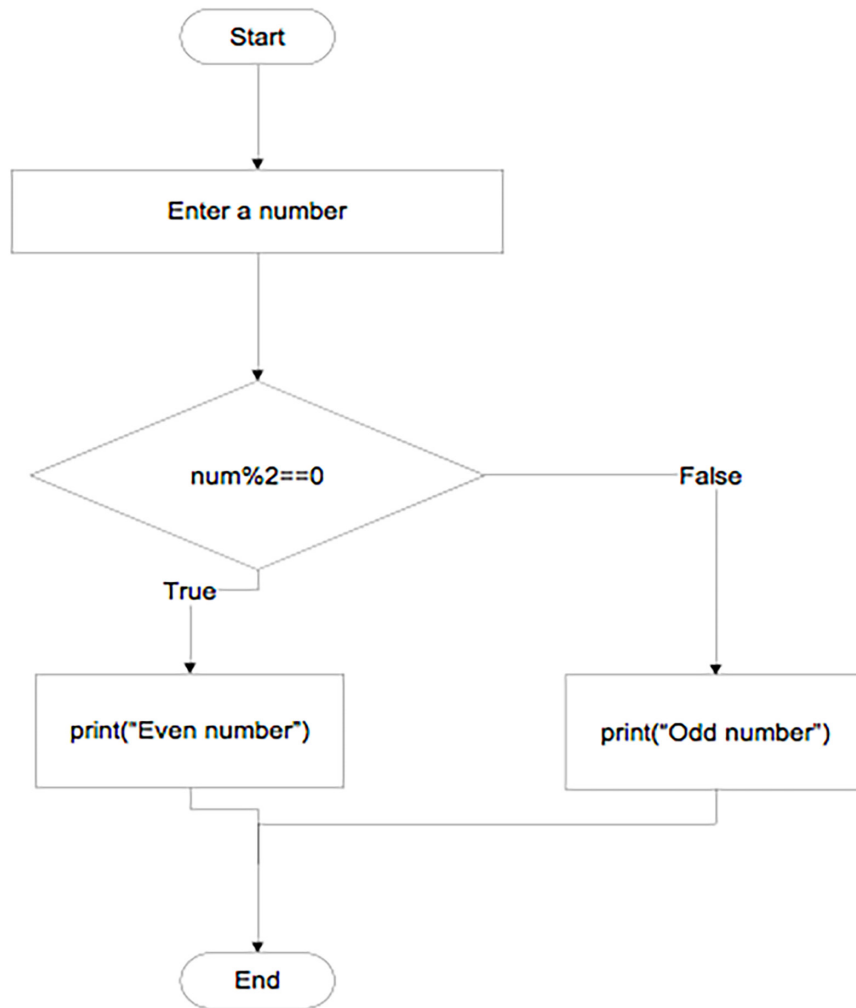


Figure 3.4: Flowchart of if-else code

if-elif-else

The word ‘*elif*’ stands for **else if**. There are cases where we are allowed to check multiple expressions.

The syntax of **if-elif-else** statement is: **if test expression:**

Body of if

elif test expression

Body of elif

else:

Body of else

80 ● Python for Everyone

From the preceding syntax, we can see that **if** block will have only one **else** block. If the test expression for **if** is True, then body of **if** is executed. If the test expression for **if** is False, then it will check the test expression of next **elif** block and so on. If the test expression of **elif** block is True, then body of **elif** block will be executed. If all the test expressions are False, then body of **else** will be executed. Among several **if-elif-else** block, only one block will be executed. The flowchart of **if-elif-else** block is shown in Figure 3.5:

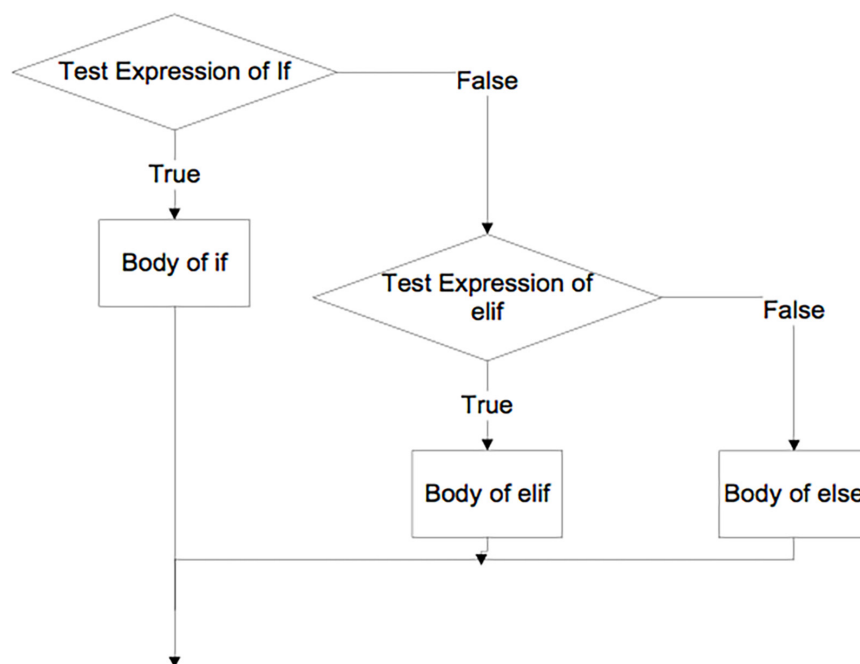


Figure 3.5: Flowchart of if-elif-else statement

Refer to the following program:

```
num1 = 9
```

```
num2 = -12
```

```
num3 = 7
```

```
if num1
```

```
print(f"1: The smallest number is {num1}")

elif num2 < num3:

    print(f"2: The smallest number is {num2}")

else:

    print(f"3: The smallest number is {num3}")
```

Concept of Flow Control Statements in Python ● 81

Output:

2: The smallest number is -12

Note: The preceding code is covered in (Program Name: Chap03_Example3.3.py)

In the preceding example, we are comparing whether the first number (**num1**) is less than **num2** and **num3**. Since the test expression is False, the control goes to **elif** block. The number 12 is less than 7. Hence, output is “**2: The smallest number is -12**”. The flow chart of the preceding code is shown in *Figure 3.6*:

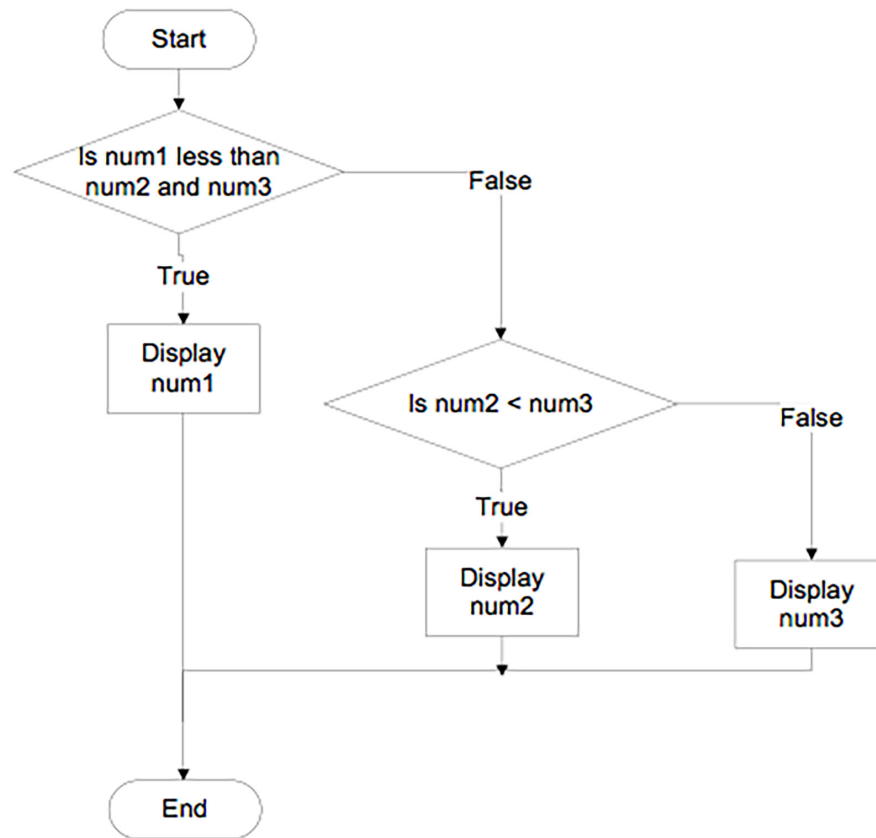


Figure 3.6: Flowchart for if-elif-else code

if-elif-else ladder

A user is allowed to choose between multiple options. These test expressions will be executed from top to down. The moment one of the **if** statement becomes True, the block of statement associated with that **if** will be executed. The final **else** block will be executed if none of the test expressions are True.

82 ● Python for Everyone

Just observe the following example:

```
my_age = float(input("Enter the age: "))
```

```
if my_age > 0 and my_age < 1.5:
```

```
print("The age is of infant")

elif my_age >=1.5 and my_age <12:

    print("The age is of children")

elif my_age >=12 and my_age <17:

    print("The age is of Teenager")

elif my_age >=17 and my_age <30:

    print("The age is of adult")

elif my_age >=30 and my_age <46:

    print("Middle aged person")

else:

    print("The age is of elder person")
```

Output:

Scene-1

Enter the age: 1.0

The age is of infant

Scene-2

Enter the age: 6.0

The age is of children

Scene-3

Enter the age: 13.0

The age is of Teenager

Scene-4

Enter the age: 20.0

The age is of adult

Scene-5

Enter the age: 35.0

Middle aged person

Scene-6

Enter the age: 47.0

The age is of elder person

Note: The preceding code is covered in (Program Name: Chap03_Example3.4.py)

In the preceding example, the user is prompted to enter the age.

In Scene-1, user has entered the age as 1.0. So, test expression of **if** is True. Hence, output “The age is of infant” is displayed.

In Scene-2, user has entered the age as 6.0. So, test expression of first **elif** is True. Hence, output “The age is of children” is displayed.

In Scene-3, user has entered the age as 13.0. So, test expression of second **elif** is True. Hence, output “The age is of Teenager” is displayed. In Scene-4, user has entered the age as 20.0. So, test expression of third **elif** is True. Hence, output “The age is of adult” is displayed.

In Scene-5, user has entered the age as 35.0. So, test expression of fourth **elif** is True. Hence, output “Middle aged person” is displayed.

In Scene-6, user has entered the age as 47.0. So, test expression of **else** part is True. Hence, output “The age is of elder person” is displayed.

84 ● *Python for Everyone*

The Flowchart of the preceding program is shown in *Figure 3.7*:

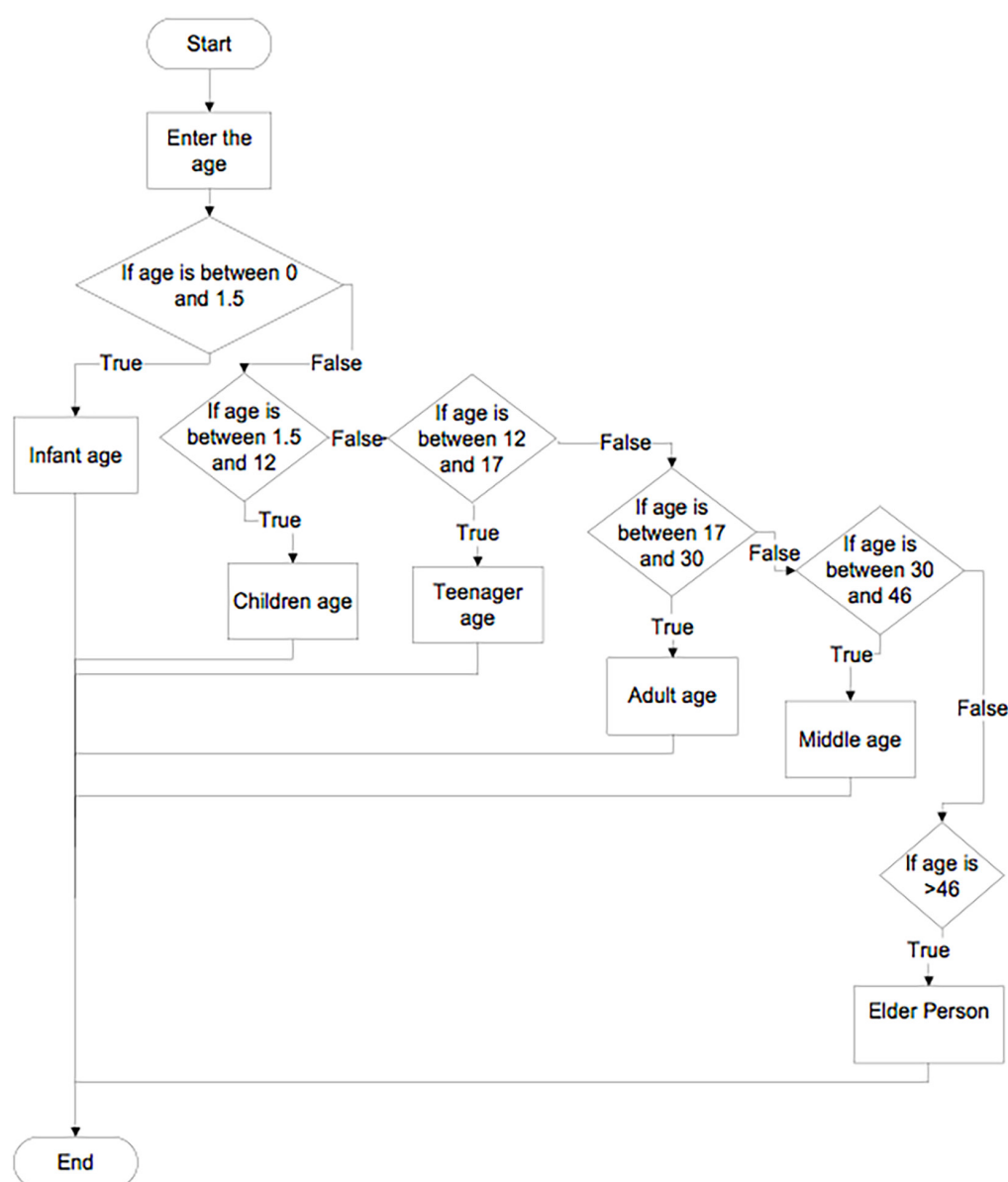


Figure 3.7: Flowchart for if-elif-else ladder code

The **else** part is always optional. It is not compulsory to write **else** statement with an **if** statement.

Moreover, there is no switch statement in Python.

Iterative statements

Sometimes a group of statements are executed multiple times or repeatedly. We then require iterative statements. They are termed as loops. As long as the condition is True, loop statements will allow us to execute a block of statements. This loop word

Concept of Flow Control Statements in Python ● 85

will be used many times during the course of Python program. Without loops, we cannot even think of making a Python code. There are only 2 loops in Python: **for** and **while** loop. There is no concept of **do-while** loop in Python.

for

The **For** loop in Python iterates over a sequence of elements. The sequence can be either string, list, tuple, set or range. To do some necessary action for every element present in some sequence, the **for** loop is used. The syntax of **for** loop is: **for val in sequence:**

Body of for

Here, **val** is a variable which takes item value inside the sequence on each iteration. Using indentation, the body of **for** loop is separated from the rest of the code. Until the last item in the sequence is reached, loop is continued. The flow chart of for loop is shown in *Figure 3.8*:

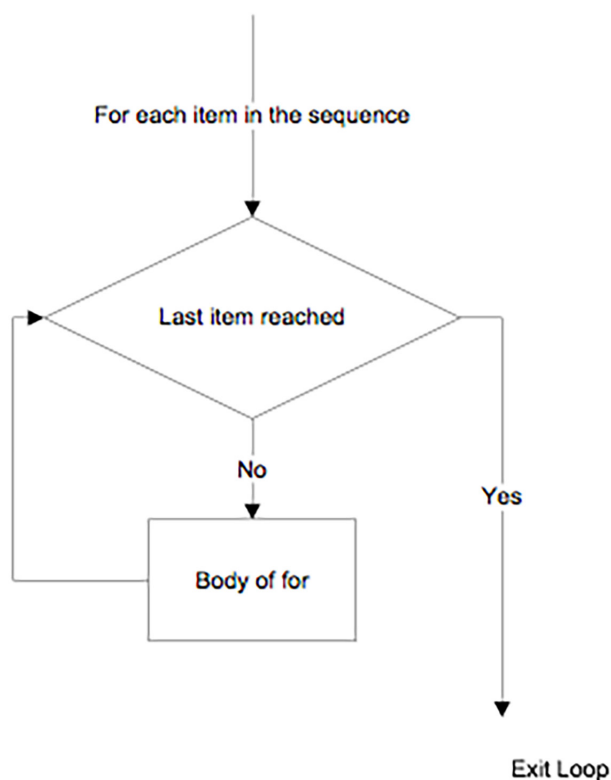


Figure 3.8: Flowchart of for loop

Let us see some solved examples of **for** loop: for i in range(4):#0 to 3

86 ● *Python for Everyone*

```
print(f"I am: {i}")# F1
```

```
for i in range(1,5):#1 to 4
```

```
print(f"I am: {i}")# F2
```

Output:

I am: 0

I am: 1

I am: 2

I am: 3

I am: 1

I am: 2

I am: 3

I am: 4

Note: The preceding code is covered in (Program Name: Chap03_Example3.5.py)

In F1, the **range()** method is used to iterate through a sequence of numbers. **range(4)** means the numbers will be iterated from 0 to 3. So, first **i = 0**, then **print()** function will be executed. So, “I am: 0” will be displayed. The loop will continue for **i = 1, 2** and **3**. Hence, the final output to be displayed is:

I am: 0

I am: 1

I am: 2

I am: 3

Similarly in F2, the **range(1,5)** will iterate from 1 to 4. Hence, final output is: I am: 1

I am: 2

I am: 3

Concept of Flow Control Statements in Python ● 87

I am: 4

We shall see one more example to find the sum of first **n** natural numbers: **num = int(input("Enter the number: "))**

total = 0

for i in range(1,num+1):

total += i

```
print(total)
```

Output:

Enter the number: 5

15

Note: The preceding code is covered in (Program Name: Chap03_Example3.6.py)

In the preceding example, user is prompted to enter the number. The number entered is 5. Initially total variable value is 0. The **range()** will iterate from 1 to 5. So, the loop will be iterating 5 times.

1st Iteration

$i = 1$

$total += 1 \Rightarrow total = 0 + 1$

$total = 1$

2nd Iteration

$i = 2$

$total += 2 \Rightarrow total = 1 + 2$

$total = 3$

3rd Iteration

$i = 3$

$total += 3 \Rightarrow total = 3 + 3$

$total = 6$

4th Iteration

`i = 4`

`total += 4 => total = 6 + 4`

`total = 10`

5th Iteration

`i = 5`

`total += 5 => total = 10 + 5`

`total = 15`

So, the sum of first 5 natural numbers is 15.

while

In **while** loop, a block of code is executed as long as the test expression is True and the control will come out of the loop if the test expression is False. At the beginning of the loop, the condition is checked every time. The body of **while** loop is determined through indentation and the first un-indented line marks the end. The syntax of **while** loop is as follows:

While test expression:

Body of while

Here, test expression is checked first. If found True, then the control enters into the body of **while** loop. After one iteration, the control goes again to evaluate test expression. This is a repetitive process until the test expression is evaluated to **False**. The flow chart of **while** loop is shown in *Figure 3.9*:

Concept of Flow Control Statements in Python ● 89

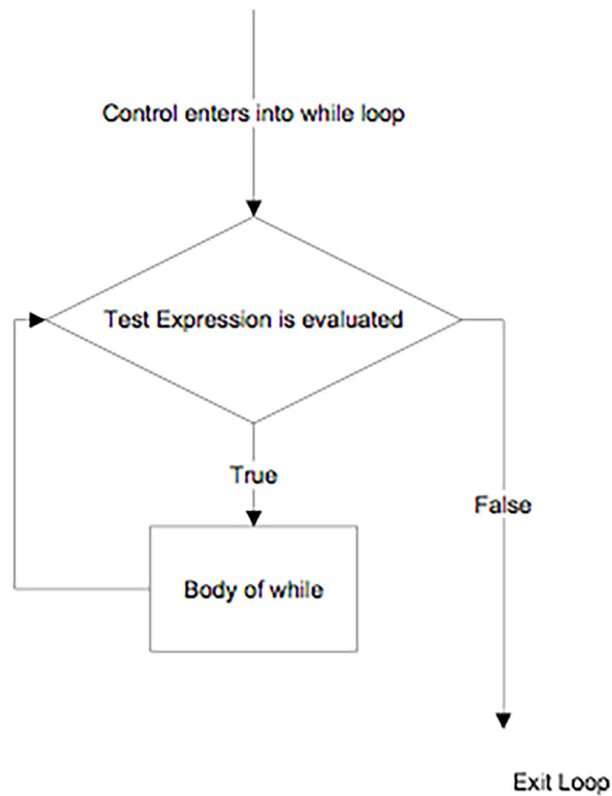


Figure 3.9: Flowchart of while loop

Let us see some examples:

```
j = 1
```

```
while j<=5:
```

```
    print("Welcome Python Beginners :)")
```

```
    j += 1
```

Output:

Welcome Python Beginners :)

Welcome Python Beginners :)

Welcome Python Beginners :)

Welcome Python Beginners :)

Welcome Python Beginners :)

Note: The preceding code is covered in (Program Name: Chap03_Example3.7.py)

In the preceding program, test expression will be True if our counter variable **j** is less than equal to 5. The most important point to observe is that the value of counter variable in the body of the **while** loop has to be incremented. Here, **j** is incremented with the value of 1. If failed to do so, it will result in an infinite loop. The code will be continuously running and has to be forcefully stop. The result is finally displayed as the string “Welcome Python Beginners :)” is displayed 5 times. We can see how the control goes during each iteration.

1st Iteration

Test Expression: True

```
j = 1
```

```
print(" Welcome Python Beginners :) ")
```

```
j = 2
```

2nd Iteration

Test Expression: True

```
j = 2
```

```
print(" Welcome Python Beginners :) ")
```

```
j = 3
```

3rd Iteration

Test Expression: True

```
j = 3
```

```
print(" Welcome Python Beginners :) ")
```

j = 4